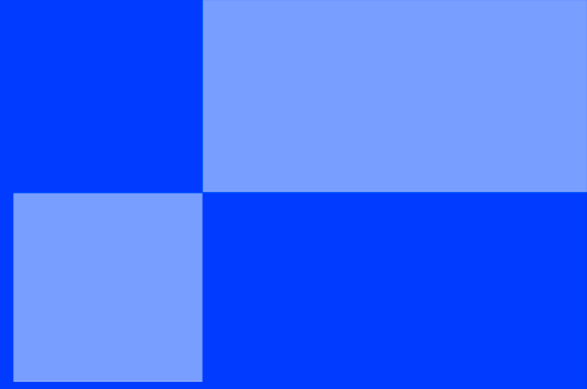




DAY 07 - BONUS TASKS

< HANGMAN: BONUS BRICKS />

DAY 07 - BONUS TASKS

EXTRA EXERCISES



Pushing the limits

Extra exercises to go further, once (or while) you build the hangman project.

They are **not** the pieces of the game itself: they train the same skills on new problems, and some of them extend your game.

Each exercise is independent. Test every function on your own with several inputs (including edge cases) before moving on.

1. Word workout

Task 1.1: Vowels and consonants

Write a function that takes a string and prints how many vowels and how many consonants it contains.

- ✓ Ignore anything that is not a letter (spaces, digits, punctuation).
- ✓ Uppercase and lowercase are treated the same.

```
count_types("Hello World!") -> 3 vowels, 7 consonants
```

Task 1.2: Palindrome

Write a function that returns `True` if a word reads the same forwards and backwards, `False` otherwise.

- ✓ Case does not matter.
- ✓ Take inspiration from yesterday version, but use a loop instead of recursion this time.

```
is_palindrome("Kayak") -> True  
is_palindrome("hangman") -> False
```

Task 1.3: Anagram

Write a function that takes two words and returns `True` if they are made of exactly the same letters (same letters, same number of times).

```
is_anagram("listen", "silent") -> True  
is_anagram("hello", "world") -> False
```

2. Building blocks

Task 2.1: Filter the dictionary

You have a list of words (for example, built from `english_word_lower_set`). Write three functions:

1. One that returns only the words of length `n`.
2. One that returns only the words made exclusively of letters (no digits, no `-`, no spaces).
3. One that returns a dict grouping the words by length.

```
{3: ["cat", "dog"], 4: ["bird"]}
```

Finally, ask the user for a length and pick a random word of that length.

Task 2.2: Letter frequency

Write a function that takes a word and returns a dict with the number of occurrences of each letter.

```
frequency("banana") -> {"b": 1, "a": 3, "n": 2}
```

Write a second function that returns the most frequent letter of a word. If several letters are tied, return the first one in alphabetical order.

Task 2.3: Hint, at a price

Add a hint system to your game.

- ✓ The player types `?` to ask for a hint.
- ✓ The game reveals one random letter that has not been found yet.
- ✓ The hint costs **2 penalties**.

Handle these cases:

- The hint is refused if the word is already fully revealed.
- The hint is refused if it would bring the player to 12 penalties or more.

Task 2.4: Scoreboard

Store the results of several games in a list of dicts:

```
history = [  
    {"word": "apple", "won": True, "penalties": 6},  
    {"word": "banana", "won": False, "penalties": 12},  
]
```

Write functions that return:

- The win rate, as a percentage.
- The average number of penalties on won games.
- The longest word the player has found.

Then plug it into your game: after each game, ask "Play again?", and print the stats when the player quits.



Think about what should happen when the list is empty, or when no game has been won.

Task 2.5: Word scramble

Create a second mini-game: the program picks a word, shuffles its letters, and the player must find the original word.

- ✓ Shuffle the letters yourself, using only what you have seen (`random.shuffle` is not allowed).
- ✓ The scrambled word must never be identical to the original.
- ✓ The player has a limited number of attempts.
- ✓ Show the number of attempts left after each try.

3. Mini-projects

Task 3.1: The bot player

Write a bot that plays hangman against your game.

At each turn, the bot knows:

- The current pattern (for example A _ _ _ E).
- The letters already tried.
- A list of possible words.

Your goal is to make it guess the letters intelligently, using only this information.

- ✓ Let the bot play 100 games in a row.
- ✓ Print its win rate.



Before coding: write the pseudocode and challenge it with a comrade, as in the *Pseudocode* brick. Can you beat your own bot by improving its strategy?

Task 3.2: Two-player mode

Player 1 types a secret word, then the terminal is "cleared" (print enough blank lines to hide it). Player 2 plays. Then the roles are swapped.

Requirements:

- Validate the secret word (letters only, at least 3 letters).
- Keep a score for each player (the fewest penalties wins the round).
- After 3 rounds, print the winner (and handle a tie).

Task 3.3: Wheel of themes

Create a dict of themes, each associated with a list of words:

```
themes = {  
    "fruits": ["apple", "banana", ...],  
    "animals": [...],  
    "code": ["python", "function", ...],  
}
```

Requirements:

- The player chooses a theme, or `random`.
- The theme is displayed as a clue during the game.
- The number of allowed penalties depends on the difficulty (theme, word length...). Define your own formula.

4. Still not satisfied?

Time for a new game. Same skills, brand new project.

Task 4.1: Tic-Tac-Toe

Make a Tic-Tac-Toe game for 2 players, playable in the terminal.

- ✓ Display the board after each move.
- ✓ The players play in turn.
- ✓ Refuse invalid moves (out of the board, cell already taken).
- ✓ Detect a win and a draw.

Task 4.2: TIC-TAC-TOE

Update your Tic-Tac-Toe game so it can be played on a board of any size. It should take the size as parameters:

```
def tictactoe(rows, cols):
```

- ✓ The board display must adapt to the size.
- ✓ Decide on a winning rule that works for any size, and explain it to the players.
- ✓ Check that your game still works for a 3x3 board.

Task 4.3: Bots everywhere

Make your game singleplayer by adding a bot opponent.

- ✓ At the start of a round, ask how many players (1 or 2).
- ✓ In singleplayer, the bot plays instead of player 2.
- ✓ Start with a simple bot, then try to make it smarter.



Can you make a bot that never loses on a 3x3 board?



Never remain stuck. Ask your fellow comrades, your local pedago, or the web.

v1

{EPITECH}